

pseudo algorithm

pseudo_algorithm — Algorithm 1

User State Update (Reliability + Experience + Anomaly)

Purpose

Update all user-dependent quantities after a new interaction:

- $R_i^*(t) \rightarrow$ reliability
- $Exp_i(t) \rightarrow$ experience
- $Anom_i(t) \rightarrow$ anomaly

Inputs

user_id = i interaction k: post_id = j vote = $s_k \in \{-1, +1\}$ timestamp = t_k correctness_label = y_j (if available later) current_time = t

Outputs

Updated user state: $\alpha_i(t)$ $\beta_i(t)$ $R_i^*(t)$ $Exp_i(t)$ $Anom_i(t)$

Algorithm

Step 1: Update Time-Decayed Reliability Counts

for each past interaction m of user i: $decay_m = \exp(-\lambda_r * (t - t_m))$ $\alpha_i = \sum(decay_m * I(correct_m))$ $\beta_i = \sum(decay_m * I(incorrect_m))$

Step 2: Compute Reliability

if $(\alpha_i + \beta_i) > 0$: $R_i = \alpha_i / (\alpha_i + \beta_i)$ else: $R_i = \text{prior_default}$

Step 3: Compute Confidence

$Conf_i = 1 - \exp(-k * (\alpha_i + \beta_i))$

Step 4: Final Reliability

$R_i_star = R_i * Conf_i$

Step 5: Update Experience

$E_i = 0$ for each interaction m of user i: $decay_m = \exp(-\lambda_E * (t - t_m))$ $E_i += decay_m * q_m$

Step 6: Normalize Experience

$Exp_i = \log(1 + E_i) / \log(1 + E_max)$

Step 7: Compute Anomaly (Rule-Based)

compute $D1 = \text{burst_deviation}(i)$ compute $D2 = \text{entropy_deviation}(i)$ compute $D3 = \text{consensus_deviation}(i)$
compute $D4 = \text{coordination_score}(i)$ compute $D5 = \text{location_inconsistency}(i)$ $\text{Anom_rule} = 1 - \exp(-(\alpha_1 D1 + \alpha_2 D2 + \alpha_3 D3 + \alpha_4 D4 + \alpha_5 D5))$

Step 8: Compute ML Anomaly (Optional)

$x_i = \text{extract_features}(i)$ $\text{Anom_ML} = \text{sigmoid}(\text{model}(x_i))$

Step 9: Final Anomaly

$\text{Anom}_i = (1 - \beta) * \text{Anom_rule} + \beta * \text{Anom_ML}$

State Update

store: α_i β_i R_{i_star} Exp_i Anom_i

Key Notes

Delayed Correctness Update

if ground_truth y_j not yet available: skip α/β update

Streaming Optimization

Instead of recomputing sums:

$\alpha_i \leftarrow \text{decay_previous} + \text{new_value}$ $\beta_i \leftarrow \text{decay_previous} + \text{new_value}$

Complexity

Time: $O(\# \text{user_interactions})$ or $O(1)$ incremental

Pseudo Insight

Update user $\rightarrow$ then everything downstream becomes more accurate

pseudo_algorithm — Algorithm 2

Graph Update and Trust Propagation

Purpose

Construct and update the user interaction graph, then compute:

$T_i(t)$

which is the network-adjusted trust score for each user.

Inputs

New interaction: user_id = i post_id = j timestamp = t_k Existing data: interactions on post j current adjacency matrix A base reliability $R_{i^*}(t)$

Outputs

Updated: adjacency matrix A_{ij} normalized matrix A_{norm} trust vector $T_i(t)$

Part A — Graph Edge Update

Step 1: Identify Co-Interacting Users

neighbors = all users who interacted with post j

Step 2: For Each Neighbor $j \in \text{neighbors}$

2.1 Compute Agreement

agree = average over shared posts: $I(s_i == s_j)$

2.2 Compute Temporal Similarity

time_sim = average: $\exp(-\text{abs}(t_i - t_j) / \tau)$

2.3 Compute Frequency Strength

freq = (# shared posts between i and j) / max_shared_posts

2.4 Combine Edge Weight

$A_{ij} = w_1 * \text{agree} + w_2 * \text{time_sim} + w_3 * \text{freq}$

2.5 Update Graph

GraphRepo.upsert(i, j, A_{ij}) GraphRepo.upsert(j, i, A_{ij})

Part B — Normalization

Step 3: Normalize Rows

for each user u : sum_weights = sum($A_{u,v}$ for all v) for each neighbor v : $A_{\text{norm}}[u][v] = A_{u,v} / (\text{sum_weights} + \epsilon)$

Result

Each row of A_{norm} sums to 1

Part C — Trust Propagation

Step 4: Initialize Trust Vector

$T_{\text{old}} = R_{\text{star}}$ # from Algorithm 1

Step 5: Iterative Update

for iteration k in 1 to K : $T_{\text{new}} = \lambda_g * (A_{\text{norm}} * T_{\text{old}}) + (1 - \lambda_g) * R_{\text{star}}$ if $\text{norm}(T_{\text{new}} - T_{\text{old}}) < \epsilon$: break $T_{\text{old}} = T_{\text{new}}$

Step 6: Final Trust

$T = T_{\text{new}}$

Part D — Store Results

for each user i : update users table: $\text{trust_score} = T[i]$

Key Notes

1. Sparse Graph Optimization

store only top-K neighbors per user

2. Incremental Updates

update only affected users after each interaction

3. Periodic Recompute

run full propagation every N minutes

Complexity

Edge update: $O(\text{neighbors})$ Propagation: $O(|E| \times \text{iterations})$

Behavior Insight

Pattern Effect trusted cluster boosts trust suspicious cluster suppresses trust isolated user trust $\approx$ reliability

Pseudo Insight

Trust spreads through the network, but is anchored to real behavior

pseudo_algorithm — Algorithm 3

Post Update (Evidence, Credibility, Variance)

Purpose

Update all post-dependent quantities after a new interaction:

- $N_j(t) \rightarrow$ effective interaction mass
- $S_j(t), S_j^+, S_j^- \rightarrow$ weighted signals
- $C_{\{\text{Bayes}\}}(j,t) \rightarrow$ probabilistic credibility
- $\text{Var}_j(t) \rightarrow$ stability measure

Inputs

post_id = j interaction k: user_id = i vote = $s_k \in \{-1, +1\}$ timestamp = t_k current_time = t user state: $w_i(t_k)$
post state: previous interactions V_j

Outputs

Updated post state: $N_j(t) S_j(t) S_{j+}(t) S_{j-}(t) C_{\text{Bayes}}(j,t) \text{Var}_j(t)$

Algorithm

Step 1: Initialize Aggregates

$S_{\text{plus}} = 0$ $S_{\text{minus}} = 0$ $N = 0$

Step 2: Iterate Over Interactions

for each interaction m in V_j : $\text{user} = i_m$ $\text{vote} = s_m$ $\text{time} = t_m$ $\text{weight} = w_{i_m}(t_m)$ $\text{decay} = \exp(-\lambda * (t - t_m))$ $\text{contribution} = \text{weight} * \text{decay}$

Step 3: Update Positive / Negative Signals

if $\text{vote} == +1$: $S_{\text{plus}} += \text{contribution}$ else: $S_{\text{minus}} += \text{contribution}$

Step 4: Update Interaction Mass

$N += \text{contribution}$

Step 5: Compute Net Signal

$S = S_{\text{plus}} - S_{\text{minus}}$

Step 6: Compute Bayesian Credibility

$C_{\text{Bayes}} = (S_{\text{plus}} + \alpha_0) / (S_{\text{plus}} + S_{\text{minus}} + \alpha_0 + \beta_0)$

Step 7: Compute Variance

$\text{Var} = 0$ for each interaction m in V_j : $\text{vote} = s_m$ $\text{weight} = w_{i_m}(t_m)$ $\text{decay} = \exp(-\lambda * (t - t_m))$ $\text{Var} += \text{weight} * \text{decay} * (\text{vote} - C_{\text{Bayes}})^2$ $\text{Var} = \text{Var} / (N + \epsilon)$

Step 8: Store Results

update post j : $N_j = N$ $S_j = S$ $S_{j+} = S_{\text{plus}}$ $S_{j-} = S_{\text{minus}}$ $C_{\text{Bayes}} = C_{\text{Bayes}}$ $\text{Var}_j = \text{Var}$

Key Notes

1. Incremental Optimization

Instead of full recomputation:

update S_{plus} , S_{minus} , N incrementally

2. Use Cached User Weights

fetch w_i from Redis

3. Numerical Stability

use epsilon to avoid division by zero

Complexity

$O(\# \text{interactions for post } j)$ or $O(1)$ incremental

Behavior Insight

Pattern Outcome many trusted positive votes high C_{Bayes} mixed votes high variance low data stabilized by priors

Pseudo Insight

Post credibility emerges from trusted, recent, and consistent signals

pseudo_algorithm — Algorithm 4

Final Credibility Integration (Bayesian + ML + Memory)

Purpose

Compute the final credibility $C_j(t)$ of a post by combining:

- interaction-based evidence
- ML-based semantic prediction
- memory-based historical similarity

Inputs

post_id = j current_time = t From Algorithm 3: $C_{\text{Bayes}}(j,t)$ From ML system: $C_{\text{ML}}(j,t)$ From memory system: embeddings of post j past posts database

Outputs

$C_j(t) \rightarrow$ final credibility

Algorithm

Step 1: Compute ML-Based Credibility

text = get_post_content(j) logit = ML_model(text) $C_{\text{ML}} = \text{sigmoid}(\text{logit} / T)$

Notes

T = temperature parameter for calibration

Step 2: Compute Embedding

$v_j = \text{embedding_model}(\text{text})$

Step 3: Retrieve Similar Posts

neighbors = top_K_similar_posts(v_j)

Step 4: Compute Similarity-Based Credibility

numerator = 0 denominator = 0 for each post k in neighbors: sim = cosine_similarity(v_j, v_k) numerator += sim * C_k denominator += sim

Step 5: Normalize Memory Component

if denominator > 0: $C_{\text{memory}} = \text{numerator} / \text{denominator}$ else: $C_{\text{memory}} = \text{default_prior}$

Step 6: Combine All Signals

$C_{\text{final}} = (1 - \alpha - \gamma) * C_{\text{Bayes}} + \alpha * C_{\text{ML}} + \gamma * C_{\text{memory}}$

Step 7: Store Result

update post j: $C_j = C_{\text{final}}$

Key Notes

1. Weight Constraints

ensure: $0 \leq \alpha + \gamma \leq 1$

2. Fallback Behavior

if ML unavailable: set $\alpha = 0$ if memory empty: set $\gamma = 0$

3. Caching

cache embeddings and ML outputs

Complexity

ML inference: $O(1)$ Similarity search: $O(\log N)$ with vector index

Behavior Insight

Scenario Dominant Component new post ML active discussion Bayesian repeated topic memory

Pseudo Insight

Final credibility is a balance of evidence, semantics, and history

pseudo_algorithm — Algorithm 5

Spatial Relevance and Urgency Computation

Purpose

Compute:

- $\text{Prox}(u, j, t) \rightarrow$ user-specific spatial relevance
- $U_j(t) \rightarrow$ post urgency score

These are required for:

- alert decision
- feed ranking

Inputs

user_id = u post_id = j current_time = t User state: location $l_u(t)$ location confidence $L_u(t)$ Post state: location l_j interactions V_j content text p_j System parameters: σ_p β_1 , β_2 , β_3 Δ_t $rate_baseline$

Outputs

$Prox(u,j,t)$ $U_j(t)$

Part A — Proximity Computation

Step 1: Compute Distance

$d = \text{haversine_distance}(l_u(t), l_j)$

Step 2: Compute Spatial Decay

$spatial_factor = \exp(- (d^2) / (2 * \sigma_p^2))$

Step 3: Apply Location Confidence

$Prox = L_u(t) * spatial_factor$

Store Result

$Prox(u,j,t) = Prox$

Part B — Urgency Computation

Step 4: Compute Keyword Score

$K = 0$ for each word w in post text p_j : $K += \phi(w)$ $K = K / total_words$

Step 5: Compute Category Score

$Cat = category_model(p_j)$

Step 6: Compute Interaction Rate

$mass_recent = 0$ for each interaction k in V_j : if $t_k \in [t - \Delta_t, t]$: $mass_recent += w_{i_k}(t_k)$ $rate = mass_recent / \Delta_t$

Step 7: Normalize Velocity

$V = 1 - \exp(- rate / rate_baseline)$

Step 8: Combine Urgency Signals

$urgency_input = \beta_1 * K + \beta_2 * Cat + \beta_3 * V$

Step 9: Final Urgency

$U = 1 - \exp(- urgency_input)$

Store Result

$$U_j(t) = U$$

Key Notes

1. Keyword Function (w)

predefined urgency dictionary OR learned weights

2. Category Model

simple classifier OR ML model

3. Time Window t

controls sensitivity to bursts

4. Weight Constraint

$$\beta_1 + \beta_2 + \beta_3 = 1$$

Complexity

$$O(\#words + \#recent_interactions)$$

Behavior Insight

Pattern Effect urgent keywords increases U high activity spike increases U normal content low U

Pseudo Insight

Urgency is driven by meaning and momentum

pseudo_algorithm — Algorithm 6

Propagation Decision (Expand Post Visibility)

Purpose

Determine whether a post j should expand its visibility radius at time t, based on:

- credibility
- evidence strength
- stability
- temporal maturity
- spatial trust

Inputs

post_id = j current_time = t Post state: $C_j(t)$ $N_j(t)$ $Var_j(t)$ creation_time = t_create interactions V_j User data (for interactions): $w_i(t_k)$ $L_i(t_k)$ System parameters: θ N_{min} σ^2 T_{min} L_{min} λ

Outputs

$$\text{Expand}(j,t) \in \{0,1\}$$

Algorithm

Step 1: Check Credibility Condition

$\text{cond_credibility} = (C_j \geq \theta)$

Step 2: Check Evidence Condition

$\text{cond_evidence} = (N_j \geq N_{\min})$

Step 3: Check Stability Condition

$\text{cond_stability} = (\text{Var}_j \leq \sigma^2)$

Step 4: Check Temporal Condition

$\text{age} = t - t_{\text{create}} \quad \text{cond_time} = (\text{age} \geq T_{\min})$

Step 5: Compute Spatial Trust $\{L\}_j(t)$

Initialize

$\text{numerator} = 0 \quad \text{denominator} = 0$

Aggregate Over Interactions

for each interaction k in V_j : $\text{user} = i_k \quad \text{time} = t_k \quad \text{weight} = w_{i_k}(t_k) \quad \text{loc_conf} = L_{i_k}(t_k) \quad \text{decay} = \exp(-\lambda * (t - t_k))$
 $\text{contribution} = \text{weight} * \text{decay}$
 $\text{numerator} += \text{contribution} * \text{loc_conf}$
 $\text{denominator} += \text{contribution}$

Compute Average

if $\text{denominator} > 0$: $L_{\text{bar}} = \text{numerator} / \text{denominator}$ else: $L_{\text{bar}} = 0$

Step 6: Check Spatial Trust Condition

$\text{cond_location} = (L_{\text{bar}} \geq L_{\min})$

Step 7: Final Decision

if (cond_credibility AND cond_evidence AND cond_stability AND cond_time AND cond_location): $\text{Expand} = 1$ else: $\text{Expand} = 0$

Step 8: Apply Expansion

if $\text{Expand} == 1$: increase propagation radius $r_j(t)$

Key Notes

1. Radius Update Strategy

$r_j(t) = \min(r_j(t) * \text{growth_factor}, R_{\max})$

2. Incremental Optimization

maintain running values for numerator and denominator

3. Fail-Fast Optimization

if any condition fails early: skip remaining checks

Complexity

$O(\# \text{interactions for spatial aggregation})$

Behavior Insight

Condition Role credibility truth filter evidence prevents early spread variance ensures consensus time avoids premature expansion location trust prevents spoofing

Pseudo Insight

Content spreads only when it is trusted, supported, stable, mature, and locally reliable

pseudo_algorithm — Algorithm 7

Alert Decision System

Purpose

Determine whether a specific user u should receive an alert for post j at time t .

This is the final output layer of the system.

Inputs

user_id = u post_id = j current_time = t From previous computations: $\text{Prox}(u,j,t)$ $C_j(t)$ $U_j(t)$ $\text{Var}_j(t)$ User state: alert history System parameters: τ_p θ_{alert} σ^2 R_{max} δ_t

Outputs

$\text{Alert}(u,j,t) \in \{0,1\}$

Algorithm

Step 1: Check Proximity Condition

$\text{cond_proximity} = (\text{Prox}(u,j,t) \geq \tau_p)$

Step 2: Check Credibility–Urgency Condition

$\text{score} = C_j(t) * U_j(t)$ $\text{cond_importance} = (\text{score} \geq \theta_{\text{alert}})$

Step 3: Check Stability Condition

$\text{cond_stability} = (\text{Var}_j(t) \leq \sigma^2)$

Step 4: Check Rate Limiting

Count Alerts in Time Window

alerts_recent = count alerts sent to user u in [t - delta_t, t]

Apply Limit

if alerts_recent >= R_max: cond_rate = False else: cond_rate = True

Step 5: Final Decision

if (cond_proximity AND cond_importance AND cond_stability AND cond_rate): Alert = 1 else: Alert = 0

Step 6: Execute Alert

if Alert == 1: send_notification(u, j) store alert in database increment alert counter for user u

Key Notes

1. Proximity as First Filter

if cond_proximity == False: return Alert = 0

2. Multiplicative Importance

C_j * U_j ensures both: credibility AND urgency

3. Rate Limiting Strategy

use sliding window OR Redis TTL counter

4. Alert Deduplication (Optional)

avoid sending same alert repeatedly

Complexity

O(1) per user-post pair

Behavior Insight

Scenario Result near + credible + urgent alert far away no alert credible but not urgent no alert urgent but not credible no alert too many alerts suppressed

Pseudo Insight

Alert only when it is relevant, important, stable, and not overwhelming

System Completion Status

At this point, you have:

Mathematical Layer

- all formulas defined
- no undefined variables
- full dependency chain complete

Algorithm Layer

- user update
- graph + trust
- post computation
- ML integration
- spatial + urgency
- propagation
- alert system

End-to-End Flow

Interaction → User Update → Graph Update ↓ Post Update → Credibility → Variance ↓ ML + Memory
Integration ↓ Spatial + Urgency ↓ Propagation Decision ↓ Alert Decision

Final Insight

You now have:

a fully specified computational system not just formulas, not just architecture, but executable logic